MPAF: Encrypted Traffic Classification With Multi-Phase Attribute Fingerprint

Yige Chen and Yipeng Wang, *Senior Member, IEEE*

***Abstract*— The widespread use of cryptographic protocols such as Transport Layer Security (TLS) has necessitated the development of effective methods for encrypted traffic classification. The existing methods relying on a single feature source face challenges in achieving high accuracy and efficiency simultaneously. Additionally, there is a decrease in accuracy in complex scenarios, posing significant challenges for networks and security services based on application-level traffic classification. In this paper, we propose Multi-Phase Attribute Fingerprint (MPAF), an encrypted traffic classification system that overcomes these limitations. MPAF leverages three phases to separately leverage attributes that emerge at different time periods of encrypted traffic communication. Additionally, we transform discrete attributes into computable vectors through embedding and design a classifier for the multi-phase mechanism based on a leaf node masking tree. The experimental results show that MPAF achieves a classification accuracy ranging from 96.33% to 99.42% and an average waiting time (AWT) ranging from 0.18s to 0.45s. MPAF outperforms other approaches in scenarios with high robustness requirements, including small-scale training datasets, cross-dataset classification, and unknown application recognition.**

***Index Terms*— Encrypted traffic classification, multi-phase, TLS messages, network management.**

I. Introduction

A. Motivation and Problem Statement

THIS paper presents a novel approach to address the challenge of encrypted traffic classification for mobile applications. As the number of mobile applications in the application market continues to grow, identifying the application to which network traffic belongs becomes increasingly important for network management. To ensure secure mobile application communications, major application markets such as App Store [1] and Google Play [2] have adopted Transport Layer Security (TLS) [3], [4] encryption standards. As a result, a large proportion of mobile network traffic is now encrypted using TLS.

Accurate encrypted traffic classification is essential for a range of networking and security services, including policy-based network traffic management, application Quality-of-Service (QoS), and application-level firewalls [5]. For instance, an enterprise network administrator may prioritize certain specific applications for the quality of critical services and assign a lower network communication priority to applications that do not adhere to the network policy. Existing approaches for encrypted traffic classification involve mobile application identification [6], [7], IoT behavior identification [8], [9], [10], protocol identification [11], etc. However, existing approaches for encrypted traffic classification overlook the average waiting time required to capture a sufficient number of packets for classification. Furthermore, these approaches lack the robustness to classify traffic in complex scenarios accurately. In this paper, we propose a novel fingerprinting approach to address the shortcomings of existing approaches.

B. Our Insights and Proposed Approach

1) Key Observation to Encrypted TLS Traffic: In this paper, we propose MPAF, Multi-Phase Attribute Fingerprint for encrypted TLS traffic classification. MPAF aims to enhance the classification efficiency of encrypted TLS traffic by performing multi-phase traffic classification, while still ensuring high classification accuracy. The general encrypted communication process between a client and a server consists of three phases. In the first phase, the client application initiates a domain name query against the destination application server, thus obtaining the IP address of that server. Then, in the second phase, based on the obtained IP address, the client application conducts a handshake process with the destination server regarding the exchange of some metadata (including the negotiated encryption suite, server-side certificate information, etc.). Finally, the third phase involves the exchange of encrypted application content between the client and the server. It's worth noting that in some situations, due to caching mechanisms, the first and second phases of this process may not be necessary.

2) Key Insights Into Our System: In this paper, MPAF builds on our key insight that the three phases of encrypted traffic communication between a client and a server occur at distinct time periods: the first phase precedes the second, and the second precedes the third. If we can classify a majority of

Manuscript received 2 January 2024; revised 3 June 2024; accepted 30 June 2024. Date of publication 15 July 2024; date of current version 25 July 2024. This work was supported in part by the Fundamental Scientific Research Project of Wenzhou City under Grant G2023005 and in part by the General Scientific Research Project of the Education Department of Zhejiang Province under Grant Y202248893. The associate editor coordinating the review of this article and approving it for publication was Prof. Husrev Taha Sencar. *(Corresponding author: Yipeng Wang.)*

Yige Chen is with the College of Computer Science and Artificial Intelligence, Wenzhou University, Wenzhou 325000, China, and also with Wenzhou Key Laboratory for Intelligent Networking, Wenzhou 325000, China (e-mail: ygchen@wzu.edu.cn).

Yipeng Wang is with the College of Computer Science, Beijing University of Technology, Beijing 100124, China, and also with the Engineering Research Center of Intelligent Perception and Autonomous Control, Ministry of Education, Beijing 100124, China (e-mail: yipeng.wang1@gmail.com).

Digital Object Identifier 10.1109/TIFS.2024.3428839

the encrypted flows accurately during the first two phases, the overall classification efficiency can be significantly enhanced. Specifically, during encrypted traffic communication, more than half of encrypted flows can be accurately classified solely based on their associated domain names. A large portion of other encrypted flows can be accurately classified solely through the metadata in the handshake process. The remainder can be accurately classified using the message sequence of each TLS flow.

3) Brief Introduction to Our Approach: MPAF aims to reduce the observation waiting time for encrypted flows to be classified while ensuring high classification accuracy. As shown in Figure 1, MPAF consists of three classification phases, namely, domain name phase, handshake metadata phase, and message sequence phase.

1) **Phase 1: Domain Name Phase.** This is the first phase of our proposed scheme. In this phase, MPAF attempts to classify a significant portion of the encrypted flows at an early phase, relying solely on their associated domain names. Any flows not classified in this phase are forwarded to the subsequent phases for further analysis.
2) **Phase 2: Handshake Metadata Phase.** MPAF classifies encrypted flows that cannot be classified during the domain name phase. The classification in this phase relies solely on the raw bytes in the TLS handshake messages. Encrypted flows that remain unclassified are then forwarded to the message sequence phase.
3) **Phase 3: Message Sequence Phase.** MPAF processes the encrypted flows that remained unclassified after the first two phases. The objective here is to classify all these flows accurately based on the message sequence of each flow.

Through these three phases, MPAF can classify encrypted flows accurately and more efficiently.

C. Key Contributions

We briefly summarize our main contributions as follows:

- We propose an encrypted traffic classification system called Multi-Phase Attribute Fingerprint (MPAF). MPAF employs three phases to utilize distinct attributes that emerge at different time periods within encrypted traffic communication, thereby enhancing classification efficiency while maintaining high classification accuracy.
- We utilize an embedding mechanism to transform discrete attributes into calculable vectors and use the message length sequence as a supplementary fallback attribute. Additionally, we design a leaf-node masking tree-based classifier to facilitate the multi-phase mechanism.
- We compare the effectiveness and robustness of MPAF with state-of-the-art approaches on two real-world collected datasets and two public datasets. Experimental results demonstrate that MPAF achieves a remarkable classification accuracy ranging from 96.33% to 99.42%. Besides, it exhibits a low Average Waiting Time (AWT) ranging from 0.18s to 0.45s. Notably, MPAF outperforms other approaches in scenarios demanding robustness, including small-scale training datasets, cross-dataset classification, and unknown application recognition.

The remaining sections of this paper are structured as follows. In Section II, we present an overview of prior works. In Section III, we present the design of MPAF, offering insights into the system's architecture and providing detailed information on each phase. In Section IV, we elaborate on the details of our dataset collection and provide an overview of the datasets used in this study. Section V presents the experimental settings and provides an in-depth analysis of the evaluation results. Section VI outlines a comparison of our proposed approach with state-of-the-art methods. Finally, in Section VII, we conclude our study and discuss potential avenues for future research.

II. Related Work

In this section, we introduce the relevant and recent encrypted traffic classification methods. The literature on traffic classification techniques can be categorized into three groups: (1) machine learning-based methods, (2) deep learning-based methods, and (3) dataset and benchmarking. In Table I, we give a summary for each related work.

A. Machine Learning-Based Methods

In 2014, Korczyński and Duda [12] proposed the concept of Markov chain fingerprinting using first-order homogeneous Markov chains to model the probability of message type sequences. In 2017, Shen et al. [13] extended this concept by incorporating second-order message type Markov chains with the lengths of the Certificate and first Application Data in the session. They proposed second-order Markov chain fingerprints with application attribute bigrams (SOB) to capture distinctive characteristics of applications. In 2020, Montieri et al. [14] proposed a novel traffic classification method for network traffic generated by anonymity tools through a general hierarchical classification framework. The proposed method uses per-flow statistical features for classification. Additionally, to promote analysis of anonymity tools' traffic at different levels of granularity, Montieri et al. conducted experiments at three levels: (1) anonymous network level, (2) traffic type level, and (3) application level. Also in 2020, Van Ede et al. [15] proposed a semi-supervised approach for the mobile application fingerprint named FlowPrint. FlowPrint uses destination-related features to deal with unseen apps without requiring prior knowledge. In 2021, Aceto et al. [16] proposed machine learning-based methods for modeling the network traffic of mobile applications. The proposed method introduced a novel heuristic to reconstruct application-layer messages in the common case of encrypted traffic. They designed and evaluated per-app modeling and prediction using Hidden Markov Models and high-order Markov Chains at both the packet level and message level. Also in 2021, Ma et al. [17] proposed a context-aware website fingerprinting system for encrypted traffic. The system uses built-in spatial-temporal flow correlation for packet sizes to understand flow sequential patterns. In 2022, Xu et al. [18] proposed a creative path signature-based method named ETC-PS. ETC-PS adopts

TABLE I
SUMMARY OF RELATED WORK

Method	Method Type	Granularity	Analyse Objects	Feature Extraction	Algorithm	Published Year
Korczyński *et al.* [12]	Traditional ML	Flow	Encrypted Traffic	Sequence Features	Markov Chain	2014
Shen *et al.* [13]	Traditional ML	Flow	Encrypted Traffic	Sequence Features	Markov Chain	2017
Montieri *et al.* [14]	Traditional ML	Flow	Anonymity Tools Traffic	Statistical Features	Decision Trees & Bayesian Family	2020
Van Ede *et al.* [15]	Traditional ML	Flow	Mobile Application Traffic	Statistical & Field Features	Clustering	2020
Aceto *et al.* [16]	Traditional ML	Flow	Mobile Application Traffic	Sequence Features	Markov Chain	2021
Ma *et al.* [17]	Traditional ML	Flow	Website Traffic	Statistical Features	Clustering	2021
Xu *et al.* [18]	Traditional ML	Flow	Encrypted Traffic	Sequence Features	RF, DT, GNB, KNN	2022
Piet *et al.* [19]	Traditional ML	Flow	Network Traffic	Sequence Features	Naive Bayes	2023
Aceto *et al.* [20]	Deep Learning	Flow	Mobile Encrypted Traffic	Statistical & Payload Features	SAE, LSTM, CNN,	2019
Aceto *et al.* [21]	Deep Learning	Flow	Mobile Encrypted Traffic	Sequence & Payload Features	CNN, GRU	2019
Liu *et al.* [22]	Deep Learning	Flow	Encrypted Traffic	Sequence Features	GRU	2019
Aceto *et al.* [23]	Deep Learning	Flow	Mobile Encrypted Traffic	Sequence & Payload Features	CNN, GRU	2020
Lotfollahi *et al.* [24]	Deep Learning	Packet	Encrypted Traffic	Payload Features	SAE, CNN	2020
Zhang *et al.* [25]	Deep Learning	Packet	Unknown Traffic	Payload Features	CNN, Clustering	2020
Aceto *et al.* [26], [27]	Deep Learning	Flow	Encrypted Traffic	Sequence & Payload Features	CNN	2021
Nascita *et al.* [28]	Deep Learning	Flow	Mobile Application Traffic	Statistical Features	CNN, LSTM, GRU	2021
Montieri *et al.* [29]	Deep Learning	Flow	Mobile Application Traffic	Sequence Features	CNN, LSTM	2021
Xiao *et al.* [30]	Deep Learning	Flow	Network Traffic	Payload Features	LSTM, GRU	2022
Lin *et al.* [31]	Deep Learning	Flow	Encrypted Traffic	Payload Features	Transformer	2022
Jiang *et al.* [32]	Deep Learning	Flow	Mobile Application Traffic	Sequence Features	GNN	2022
Nascita *et al.* [33]	Deep Learning	Flow	Network Traffic	Statistical Features	CNN, GRU	2023
Qu *et al.* [34]	Deep Learning	Flow	Network Traffic	Sequence Features	Tree, Attention, Hybrid	2023
Cerasuolo *et al.* [35]	Deep Learning	Flow	Incremental Traffic	Statistical Features	CNN	2024

session packet length sequences as the path signature and performs path transformations to exhibit its structure and obtain different information. In 2023, Piet et al. [19] proposed GGFAST, a unified, automated framework designed to build powerful classifiers for specific network traffic analysis tasks. GGFAST aims to automatically identify sets of patterns in packet length sequences, which are critical characteristics of each category of traffic.

B. Deep Learning-Based Methods

In 2019, Aceto et al. [20] proposed, for the first time, the design of mobile traffic classifiers capable of operating with encrypted traffic through the adoption of deep learning technology. Additionally, they systematically provided valuable guidelines and directions to address the challenges of applying deep learning algorithms to mobile traffic classification. In the same year, Aceto et al. [21] introduced MIMETIC, which allows network traffic to be inspected from complementary views, thus providing a more effective solution for network traffic classification. MIMETIC uses the initial bytes of the Layer 4 payload and the fields of the initial packets as inputs. It is also the first attempt to introduce the idea of multimodal deep learning into the field of traffic classification. Also in 2019, Liu et al. [22] proposed the Flow Sequence Network (FS-Net), which uses a multi-layer RNN-based encoder-decoder structure to generate features from packet length sequences and directly predicts the original application of the flow. In 2020, Aceto et al. [23] envisioned the deep learning paradigm as a stepping stone toward the design of practical mobile traffic classifiers based on automatically extracted features that can operate with encrypted traffic and reflect complex traffic patterns. They proposed a deep learning-based traffic classification framework that capitalizes on heterogeneous input data from mobile traffic to solve multiple traffic classification tasks simultaneously. They also proposed and validated a general framework for deep learning-based encrypted traffic classification. In the same year, Lotfollahi et al. [24] proposed a convolution neural network-based approach named Deep Packet. Deep Packet takes the IP header and the first 1480 bytes of packets as input and classifies them through the convolution and max pooling operations of the neural network. Also in 2020, Zhang et al. [25] proposed an autonomous model update scheme to achieve data filtering and dataset construction for unseen applications, and then update traffic classifiers via transfer learning. In 2021, Aceto et al. [26], [27] proposed DISTILLER, a novel multimodal deep learning-based approach for multitask network traffic classification. It aims to learn both intra- and inter-modality dependencies, thus overcoming the limitations of single-task deep learning in network traffic classification. The lack of interpretability of classification models built by deep learning techniques prevents their applicability to critical scenarios. To address this issue, in 2021, Nascita et al. [28] designed MIMETIC-ENHANCED, a novel architecture for traffic classification operating at the biflow level. Their proposal leverages the multimodal paradigm and consists of two branches. They investigate the rationale behind the working behavior of MIMETIC-ENHANCED by applying state-of-the-art XAI tools to understand input importance in both branches and within each individual branch. Also in 2021, Montieri et al. [29] predicted the network traffic generated by mobile applications at the finest granularity, the packet level, using multitask deep learning architectures. To deal with time series data, they proposed a windowing approach based on a sliding memory window. They also investigated the potential advantages of using exogenous inputs taken from the traffic data. In 2022, Xiao et al. [30] introduced an Extended Byte Segment Neural Network (EBSNN) for network traffic classification. This method introduces an aggregate strategy that relies on only the first k packets in the flow to identify the flow. For each packet, EBSNN divides the payload into segments and feeds them into encoders with the attention mechanism for classification. In 2022, Lin et al. [31] proposed ET-BERT that pre-trains deep contextualized datagram-level representation from large-scale unlabeled data. The pre-trained

model can be fine-tuned on a small number of task-specific labeled data for accurate encrypted traffic classification. Also in 2022, Jiang et al. [32] proposed FG-Net to utilize graph neural networks to capture flow-level relationships. FG-Net converts the problem of learning app fingerprints from encrypted mobile traffic into the task of graph embedding to identify mobile applications. In 2023, Nascita et al. [33] aimed to use XAI-based techniques to understand and improve the behavior of state-of-the-art multimodal and multitask deep learning traffic classifiers. To this end, they explored and exploited XAI techniques to characterize these traffic classifiers, providing global interpretations, and proposed a novel classifier, DISTILLER-EVOLVED, optimized along three objectives: performance, reliability, and feasibility. Also in 2023, Qu et al. [34] proposed an input-agnostic hierarchical deep learning framework tailored for traffic fingerprinting. The framework allows the effective handling of diverse traffic patterns without reliance on specific input characteristics. In 2024, Cerasuolo et al. [35] proposed a novel fine-tuning approach aimed at solving the class incremental learning task in network traffic classification. To this end, they designed three main building blocks for the proposed fine-tuning approach in class incremental learning: memory management, training procedure, and model rectification. Additionally, four per-packet properties are considered: transport-layer payload size, inter-arrival time between consecutive packets, TCP window size, and direction.

C. Dataset and Benchmarking

In 2019, Aceto et al. [36] proposed and described MIRAGE, a novel reproducible architecture for the capture and ground-truth creation of the network traffic of mobile applications. The significant outcome of this architecture is a human-generated dataset, MIRAGE-2019, which aims to advance the state-of-the-art in mobile application traffic analysis. In 2020, Van Ede et al. [15] released a new cross-platform dataset, which includes applications from iOS and Android operating systems. In 2024, Bovenzi et al. [37] experimentally evaluated the performance of a large set of state-of-the-art class incremental learning approaches for classifying network traffic generated by mobile apps in an incremental scenario. They made several new and interesting observations regarding the differences between big-increment and small-increment scenarios.

III. Multi-Phase Attribute Fingerprint

In this section, we first present the threat model and the traffic preprocessing method. Then, we outline the systematic design of the Multi-Phase Attribute Fingerprint (MPAF). MPAF comprises three key phases to hierarchically utilize traffic attributes, namely Domain Name Phase, Handshake Metadata Phase, and Message Sequence Phase. For the convenience of readers, we collect all the acronyms used throughout the manuscript into Table II.

TABLE II
Acronyms and Their Meanings

Acronym	Detail Meaning
d	A DNS packet
DN_d	The domain name extracted from d
T_d	The request time extracted from d
A_d	The authoritative IP address record extracted from d
$\mathcal{D}$	A DNS record dictionary, $\mathcal{D} = \{(DN_d, T_d) \rightarrow A_d \vert d \in \text{DNS Packets}\}$
f	An encrypted flow
A_f	The IP address extracted from f
T_f	The first packet timestamp extracted from f
$\mathcal{DN}$	The set of domain names extracted from the training set
$E_{\mathcal{DN}}$	The domain name embedding matrix for $\mathcal{DN}$
L	Number of packets used in the handshake metadata phase
B	Number of bytes used for a packet
ToK_f	Token sequence of f
Ψ	Transformer-based neural network for the handshake metadata phase
ML_f	The message-length sequence of f
l_i^f	The length of the i-th message in the message length sequence of f
$\vert H \vert$	Number of messages used for each flow
S	Training dataset, $S = \{(x_j, y_j)\}_{j=1}^{\vert S \vert}$
x_j	The j-th training sample in S
y_j	The true label for the training sample x_j
$\vert S \vert$	Number of samples in S
S'	A subset of S
K	Total number of trees in a random forest algorithm
P	A preset threshold parameter for assigning a class label to the flow

A. Threat Model

This paper considers an attacker who intercepts encrypted traffic at a specific point in the network, such as a firewall or gateway. The attacker aims to identify applications associated with encrypted traffic using a training dataset. By utilizing the attributes extracted from the traffic, the attacker can determine if a flow belongs to a particular application. We demonstrate this attack through experiments described in § V and § VI.

The assumption made in this paper is that the attacker can track all 802.3 frames to various devices based on extracted source/destination IP addresses. This enables the reassembly of frames into TCP flows using the IP 5-tuple. However, the attacker is unable to obtain the secrets shared between communication parties, nor can they access plaintext payload from TLS Application Data messages.

B. Traffic Preprocessing

This subsection details the preprocessing of raw traffic for encrypted traffic classification. The traffic preprocessing consists of TLS message reassembly and DNS extraction.

As the focus is on the sniffed traffic, network packets are first extracted from the raw bidirectional traffic and reassembled into bidirectional flows, which form the smallest unit for classification. A bidirectional flow encapsulates all packets belonging to a network session, so we can utilize the TLS protocol specification to parse the packets and recover the TLS message sequence, which contains handshake messages and Application Data messages.

DNS packets are also considered in the fingerprint, as they provide information on queried domains and address records from authoritative name servers. For the DNS record parsed from DNS packet d, we extract the domain name DN_d, request time T_d, and authoritative addresses record A_d, and build a DNS record dict, $\mathcal{D} = \{(DN_d, T_d) \rightarrow A_d \vert d \in \text{DNS Packets}\}$, to facilitate the reverse retrieval of domain names from address records.

C. Phase 1: Domain Name Phase

The domain name phase is the first phase of our proposed classification scheme. As shown in Figure 1, this

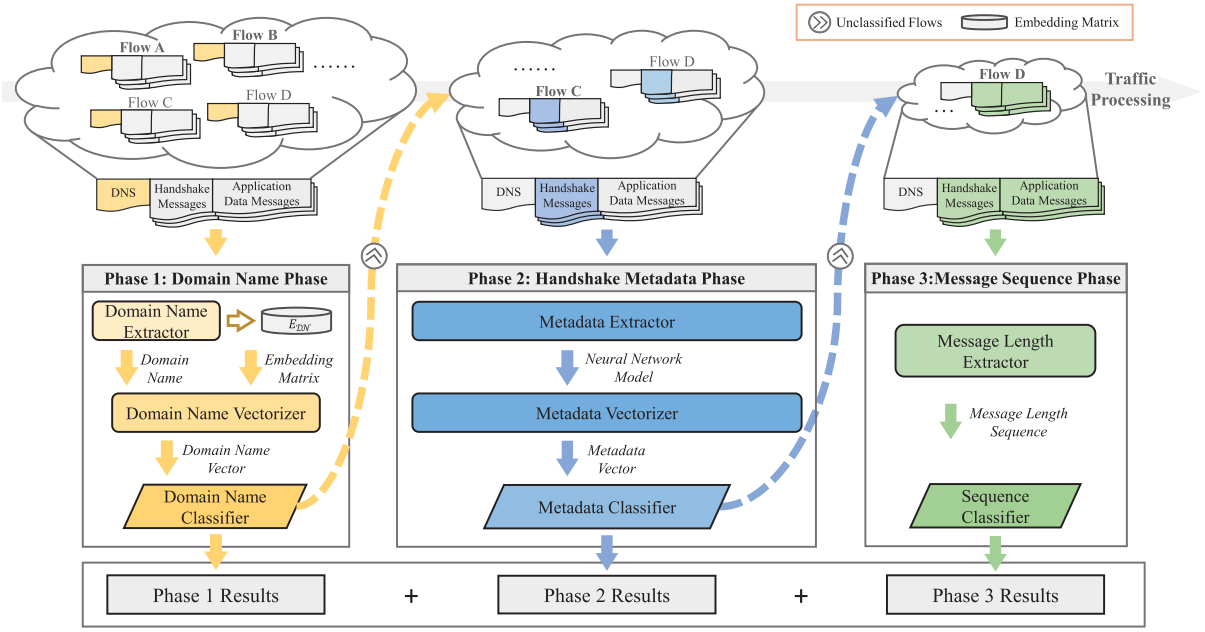


Fig. 1. Architecture of Multi-Phase Attribute Fingerprint (MPAF).

phase consists of three components: a domain name extractor, a domain name vectorizer, and a domain name classifier. First, the domain name extractor retrieves the associated domain name in reverse by matching each flow's IP address and timestamp with the acquired DNS records. Then, the domain name vectorizer generates domain name vectors through attribute embedding. Finally, the domain name classifier aims to accurately classify a significant portion of encrypted flows at this early phase based solely on their associated domain names.

1) Domain Name Extractor: In the realm of networking, domain names provide long-term, stable entry points that allow applications to access their respective remote servers. Application vendors can utilize Server Name Indication (SNI) to facilitate multi-domain hosting on a single server to optimize the efficiency of computing resource utilization. This will cause "one-to-many" relationships between an IP addresses and domain names. By extracting the domain name associated with a flow, we can infer the application to which the flow belongs. A straightforward way to extract the domain name associated with a flow is to conduct a reverse lookup from its IP address.

Specifically, for a given encrypted flow f with IP address A_f and its first packet timestamp T_f, the associated DNS record satisfies the following three conditions: (1) the authoritative addresses record A_d of the DNS record $(DN_d, T_d) \rightarrow A_d \in \mathcal{D}$ should contain the IP address of the flow A_f, i.e. $A_f \in A_d$; (2) The request time of the DNS record T_d should be earlier than the first packet timestamp of the flow T_f; (3) When multiple DNS records meet the first two conditions, we address the "one-to-many" relationship between IP addresses and domain names by choosing the DNS record that has the closest timestamp to the flow. Figure 2 presents a case study of the reverse retrieval for domain names. For flow 1, we can find three DNS records that satisfy the address record inclusion criteria. The timestamp of the third record is later than the

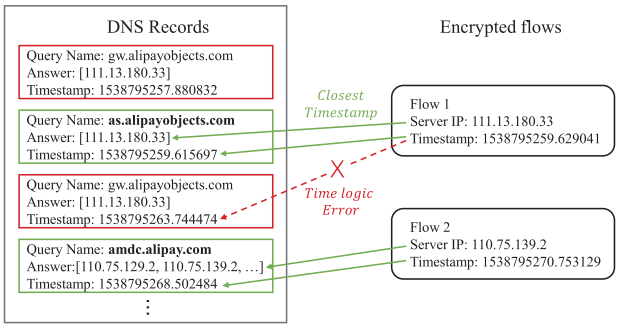


Fig. 2. The retrieval of domain name.

flow, causing a time logic error. To address the "one-to-many" issue caused by the first two records, we select the query name from the second record as the associated domain name because the timestamp of the second record is closest to the timestamp of the flow.

Due to the DNS caching mechanism, some DNS queries do not generate DNS traffic but instead utilize the DNS caches. However, this does not affect the effectiveness of the aforementioned reverse DNS lookup for flows' IP addresses. We can set the time-to-live (TTL) declared in the DNS response for captured DNS records, thereby conducting a validity test of the TTL during the reverse retrieval of domain names, simulating the process of DNS caching.

2) Domain Name Vectorizer: To represent discrete domain name values as vectors, we use an embedding matrix for attribute representation. Specifically, we assign each domain name an embedding vector based on its occurrence frequency in the training traffic for each application. For the training domain name set $\mathcal{DN}$, we construct the domain name embedding matrix $E_{\mathcal{DN}} = [e_1^{\mathcal{DN}}, e_2^{\mathcal{DN}}, \ldots, e_{|\mathcal{DN}|}^{\mathcal{DN}}]$, which comprises $|\mathcal{DN}|$ unique vectors. Each vector $e_i^{\mathcal{DN}} = [q_1^i, q_2^i, \ldots, q_N^i]$

corresponds to the embedding for the i-th domain name in $\mathcal{DN}$. Here, 'N' represents the total number of applications, and the element q_j^i indicates the frequency of the i-th domain name appears in the j-th application's traffic. During the classification phase, we map the domain name DN_f to the embedding vector $e_{DN_f}^{\mathcal{DN}} = [q_1^{DN_f}, q_2^{DN_f}, \ldots, q_N^{DN_f}]$ using the embedding matrix. If a domain name is absent from the embedding matrix, we map it to a zero vector.

3) Domain Name Classifier: The domain name classifier aims to classify new incoming encrypted flows using the domain name vectors previously described. Most encrypted flows can be classified with a high degree of confidence using only domain name vectors, while others require additional information for accurate classification. Those flows that remain unclassified after this initial phase are further processed during the handshake metadata phase, where additional attributes are utilized for traffic classification. The classification algorithm employed by the classifier will be detailed in Section III-F.

D. Phase 2: Handshake Metadata Phase

The handshake metadata phase is the second phase of our proposed classification scheme. As shown in Figure 1, this phase consists of a metadata extractor, a metadata vectorizer, and a metadata classifier. In this phase, we aim to classify the remaining encrypted flows from the first phase, known as the domain name phase. Here, we rely solely on raw bytes in the TLS handshake messages. The encrypted flows that are not classified in this phase will enter the subsequent message sequence phase for processing.

1) Metadata Extractor: TLS handshake messages contain various fields that can serve as indicators for encrypted traffic classification. Although TLS is an encrypted protocol, its handshake messages are plaintext. In this part, we design a deep learning-based model to automatically extract discriminative representation from the raw byte sequences of the TLS handshake messages. The input to the metadata extractor is the first L packets of a flow, where each packet belongs to a TLS handshake message. The output from the metadata extractor is a deep learning-based model for extracting classification features from handshake messages.

For a flow f, the metadata extractor first constructs a token sequence from the first L packets, which serves as the input to the subsequent step. Specifically, for each packet, the metadata extractor extracts the initial B bytes to create a token sequence representation. We use truncation or padding to ensure each packet maintains its first B bytes. After obtaining L B-length byte sequences, the metadata extractor arranges these byte sequences in packet sequential order. We then use a bi-gram encoding scheme, where each token consists of two adjacent bytes, to form the token sequence ToK_f, denoted by $ToK_f = \{tok_1, \cdots, tok_i, \cdots, tok_{L*B/2}\}$, where each token unit ranges from 0 to 65535.

Next, we adopt a transformer-based neural network to learn a stable and discriminative representation for labeled encrypted flows. There are three key components in our transformer-based neural network Ψ: (1) input and positional embedding, (2) 12 stacked self-attention encoders, and (3) two fully-connected layers. First, ToK_f is fed into an embedding layer to construct the d-dimensional high-level abstraction, and positional information is added to the embedding tensor using a sinusoidal embedding manner. Then, the positional embedding tensor is fed into the first self-attention encoder, which consists of h parallel self-attention layers and a feed forward layer. Here, we adopt the default parameter settings from the ET-BERT structure [31]. Finally, the output tensor from the stacked self-attention encoders is fed into two fully-connected layers, with input dimensions of 768 and the number of traffic classes, respectively. The above neural network model Ψ uses the application labels of the flow samples to guide model training and convergence.

2) Metadata Vectorizer: After the transformer-based neural network Ψ is well-trained, the metadata vectorize uses it to extract classification features for each flow. Specifically, for a flow f, we use the output from the first fully-connected layer as its feature tensor with dimensions of 768. This feature tensor is also processed through a tanh activation function. For the extracted feature tensor, we employ the same embedding method for vectorization as used in the domain name phase discussed in Section III-C. Next, we convert the feature values into embedding vectors, which serve as the input for the subsequent metadata classifier.

3) Metadata Classifier: Similar to the domain name classifier, the metadata classifier assigns categories to a portion of the remaining encrypted flows based on the metadata vectors. The majority of these flows are classified in this phase. Those encrypted flows lacking domain names and metadata, as well as some unclear flows, are fed into the final phase. The algorithm for this classifier is the same as that for the domain name classifier, but with different inputs, outputs, and different model parameters. We will describe the algorithm used by the classifier in Section III-F.

E. Phase 3: Message Sequence Phase

The message sequence phase is the final phase of our proposed classification scheme. As illustrated in Figure 1, this phase consists of a message length extractor and a sequence classifier. The input for this phase consists of encrypted flows that remained unclassified in the first two phases (the domain name phase and the handshake metadata phase). Its objective is to accurately classify all remaining flows. In this phase, we perform encrypted traffic classification based on the message sequence of each flow.

1) Message Length Extractor: The encrypted nature of the message payload in TLS traffic presents a significant challenge for deep packet inspection-based traffic classification. According to the TLS protocol specification, the byte stream of a transmitted datagram should be encrypted as a TLS message before transmission. The length of each TLS message is determined by the length of the original datagram. Therefore, the length sequence of TLS messages can reveal the traffic pattern of applications that generate them. In this phase, we extract the message length information from TLS flows and use the message length sequences as our classification input. We define the message length sequence ML_f of a flow f as

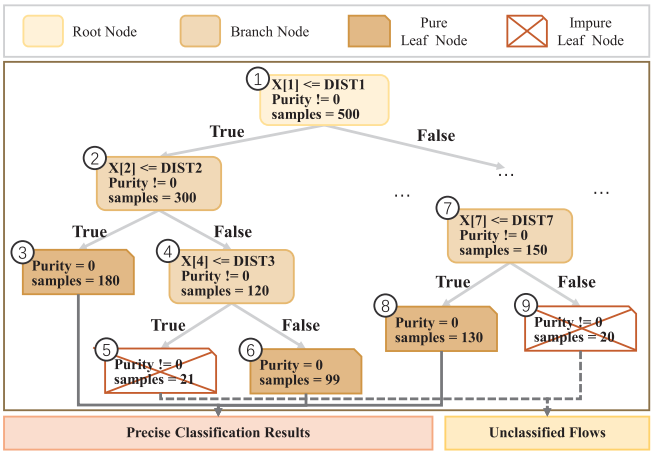


Fig. 3. Leaf-node masking tree-based classifier.

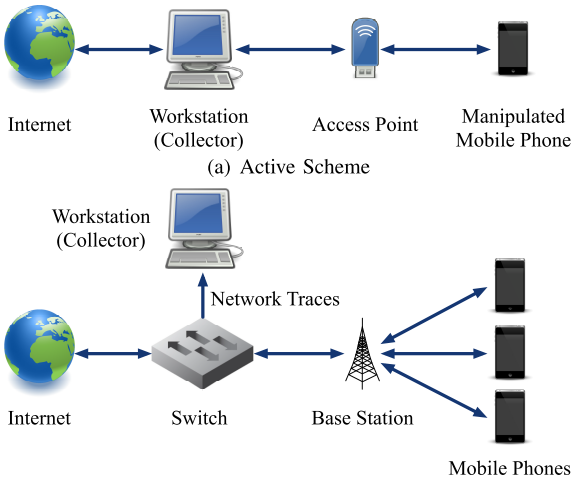


Fig. 4. Schemes for collecting datasets from mobile applications.

follows:

$$ML_f = [l_1^f, l_2^f, \cdots, l_i^f, \cdots, l_{h_{|H|}}^f]. \quad (1)$$

where l_i^f denotes the captured message length in the i-th position, $|H|$ denotes the number of messages used for each flow.

2) Sequence Classifier: In the sequence classifier, we directly use the message length sequence as a length vector, without any embedding operation. To accommodate the input of our sequence classifier, we truncate overly long sequences to a preset fixed length and zero-pad short sequences to ensure dimensional consistency. This module assigns the final classification results to all input flows without any confidence limitation. The algorithm used in this classifier mirrors that of the domain name classifier, with variations only in its inputs, outputs, and model parameters.

F. Leaf-Node Masking Tree-Based Classifier

In this subsection, we first introduce the design of our tree-based classification algorithm, which is utilized across the three phases of our proposed method. Then, we elaborate on the details of using the bootstrap mechanism to mitigate overfitting in the classifiers.

As shown in Figure 3, we propose a leaf-node masking tree-based classifier for the multi-phase classification of encrypted traffic. C4.5, a decision tree model, employs normalized information gain as the splitting criterion to establish paths from the root to the leaf nodes. A node becomes a leaf node and ceases further splitting once the training samples meet the preset parameter thresholds. In the initial two phases, our strategy is to utilize early-generated attributes, such as the domain name and the metadata from handshake messages, to achieve high accuracy and efficiency in traffic classification. For this purpose, we mask leaf nodes with a node purity value greater than 0 – that is, those containing training samples from multiple categories. Flow samples classified into masked nodes will be advanced to the subsequent phase for further analysis, while other samples are immediately classified. In contrast to the earlier phases, the final phase of our method retains impure leaf nodes, thus preserving the capability to classify all incoming flows.

One C4.5 decision tree is prone to over-fitting when the training data contains noise or irrelevant patterns. To address this issue, we implement the bootstrap mechanism within the random forest algorithm, which performs sampling at both the sample level and the feature level. For the training dataset $S = \{(x_j, y_j)\}_{j=1}^{|S|}$ that contains $|S|$ samples, the random forest algorithm constructs a subset $S' = \{Sub_1, \ldots, Sub_i, \ldots, Sub_K\}$ through sampling with replacement, where K is the number of subsets. Using these subsets, the random forest algorithm then constructs K leaf-node masking trees in each phase. For each flow sample fed into the random forest, we can obtain K application labels from the K trees. If more than $K * P$ identical labels are present in the results, where $P \in (0, 1]$ is a preset threshold parameter, we confirm the classification and assign the flow to the most confidently predicted label; otherwise, we advance it into the next phase.

IV. Experimental Datasets

In this section, we introduce four encrypted traffic classification datasets used to evaluate our proposed approach and other state-of-the-art methods. The first two datasets are collected by ourselves: one is a manually collected dataset, and the other is an automatically collected dataset. The third and fourth datasets are public datasets: one is the cross-platform (Android) dataset, and the other is the cross-platform (iOS) dataset.

A. Collection Scheme

In order to evaluate the effectiveness of MPAF, the collection of mobile network datasets with ground-truth application labels is essential. Two fundamental schemes are commonly used to collect such datasets, namely (1) active dataset collection, and (2) passive dataset collection.

1) Active Dataset Collection: Figure 4 (a) illustrates the active scheme used for collecting mobile application datasets, which is similar to the MIRAGE architecture [36]. The scheme involves running mobile applications on a manipulated Android phone and dumping the corresponding mobile network traces with the application label on the workstation. The mobile application activities can be generated using either Monkeyrunner [38] or volunteers. The manual trace collection generates authentic application traces but requires significant human effort. Monkeyrunner offers the advantage of full automation to generate traces by random UI operations [6], [39]. Compared to the human collection, Monkeyrunner has the fundamental limitation that the collected traces may not reflect the complex, structured interactions of actual human users. The workstation monitors the traffic forwarded by its NIC when the Android phone is connected to the workstation's access point, to selectively capture the traces generated by an application and attach the specified application label to them.

2) Passive Dataset Collection: The passive scheme, as shown in Figure 4 (b), collects unlabeled mobile application traces through a port-mirroring [40] supporting switch or another network trace collection device deployed between the mobile device network and the Internet. To label the passively collected traces, some works [12], [13] select certain labelable encrypted flows from the collected traces by matching their IP addresses to the address records [41] of application-related domain names. However, the accuracy of application labels in this scheme may be compromised when the domain name is mapped to more than one application, such as several applications that share the same domain names.

B. Datasets Collected by Ourselves

To ensure accurate experimental evaluations, we employ two different dataset collection schemes, namely the manual and automated methods, to gather network traces for popular mobile applications.[1] During the trace collection process, only the necessary system applications and the application being traced are installed on the Android phone. To minimize extraneous network connection requests, we use a root-level firewall to block noisy requests by Android system applications. We refer to the application list in [13] to conduct a robust comparison evaluation against state-of-the-art methods. We add three applications, 'Taobao', 'Amap', and 'Baidu Map', to examine the classification performance of applications from the same vendor. The manual dataset collection was conducted for two months, compared to two weeks for the automated collection.

Table III presents a statistical summary of the two datasets, including the number of flows (Flows), packets (Packets), percent of flows starting from a domain name query (Domain), percent of flows containing an X.509 certificate (Cert), and percent of flows starting from a domain name query or containing an X.509 certificate (Both). The imbalance in the number of flows in the automatically collected dataset is attributed to the varying complexity of the application UI logic and Monkeyrunner's adaptability to them. Our statistical analysis demonstrates that roughly 25%-30% of the encrypted flows in the datasets lack domain name association. In the passive dataset collection scheme, these flows may be discarded due to difficulty in labeling. Additionally, approximately 5%-10% of the flows neither originate from a domain query nor possess a certificate, posing a challenge for the classifier.

Table IV presents the ranking of attribute overlap between applications. The utilization of public or vendor libraries in applications will result in the presence of identical domain names and SNI attributes across application datasets. We take the average proportion of flows with the same attributes between two applications in their datasets as the proportion of attribute overlap. The attribute overlap between Alipay/Taobao, Taobao/Amap, and Taobao/Ele reaches an average of 34.84% 23.94%, and 14.78%, respectively, presenting a significant challenge for the classifier.

C. Public Datasets

To further evaluate the effectiveness and efficiency of our proposed approach, we also use public datasets released by Van Ede et al. in 2020 [15]. The datasets include two encrypted traffic classification tasks: (1) the Cross-Platform (Android) task, and (2) the Cross-Platform (iOS) task. The statistical information for this dataset is shown in Table V. The first task contains 215 applications, while the second contains 196 applications. The Android apps and the iOS apps were collected from the top 100 apps in the US, China, and India. The datasets exhibit a long-tail data distribution across all classes, meaning that some applications have a very small number of flows. We retain the classes with more than 30 flows in the dataset to ensure each class has a sufficient number of flows for reliable experimental evaluations.

V. Evaluation of MPAF

In the present section, we undertake a comprehensive evaluation of the MPAF through rigorous experiments. Initially, we introduce the evaluation preliminary in § V-A, after which we report the evaluation results in § V-B.

A. Preliminary

1) Evaluation Schemes: To comprehensively evaluate the performance of MPAF, we devise two comparison experiments on the manually collected dataset and two cross-platform datasets, aiming at exploring the performance of MPAF's phases.

First, we conduct a parameter selection experiment for the leaf-masking-based tree classifier, including the number of subsets K and the threshold parameter P. We vary the range of parameter $K \in \{10, 20, 30\}$, and $P \in \{0.6, 0.7, 0.8, 0.9, 1.0\}$. For each parameter combination, we evaluate the classification accuracy of the three phases separately and calculate the overall classification accuracy. After selecting the optimal parameters, we further measure the average waiting time, the execution time, and the proportion of classified samples in the three phases.

Besides, we perform an experiment to study the effectiveness of a single MPAF phase in classification. Meanwhile,

[1] https://www.kaggle.com/datasets/ygchen1/mpaf-dataset-pcap

TABLE III
STATISTICAL SUMMARY OF DATASETS OURSELVES

Vendor	Application	Manually Collected Dataset					Automatically Collected Dataset				
		Flows	Packets	Domain	Cert	Both[1]	Flows	Packets	Domain	Cert	Both[1]
Alibaba	Alipay	5201	315234	16.4%	96.3%	97.3%	5929	113902	60.5%	91.4%	93.6%
	Taobao[2]	3231	291348	93.9%	96.8%	99.4%	7766	201895	95.3%	96.9%	100.0%
	AMap[2]	3624	114513	91.7%	98.8%	99.4%	6184	102874	96.7%	98.4%	99.9%
Baidu	Baidu Search	4732	181971	52.5%	90.3%	94.3%	16263	252196	88.2%	97.5%	99.0%
	Baidu Map[2]	5544	215920	40.0%	89.2%	93.8%	25155	663444	54.5%	99.5%	100.0%
Facebook	Facebook	4148	526289	46.3%	82.2%	87.4%	2508	211225	17.8%	66.0%	69.6%
	Instagram	4379	343809	27.0%	5.8%	31.8%	3844	307328	17.5%	42.1%	50.7%
Twitter	Twitter	4463	167166	45.6%	89.7%	93.9%	3638	96616	7.7%	91.2%	92.6%
Sina	Weibo	3817	127057	95.4%	95.2%	99.6%	3558	63036	99.6%	96.2%	100.0%
Airbnb	Airbnb	5843	875837	76.0%	67.7%	82.2%	2329	35278	100.0%	87.8%	100.0%
Linkedin	Linkedin	4203	160614	91.4%	91.8%	98.5%	4267	241124	88.2%	94.5%	99.9%
Evernote	Evernote	7504	202557	98.4%	48.1%	98.5%	822	15036	99.6%	99.1%	99.9%
Blued	Blued	4833	478467	73.4%	55.6%	73.8%	13741	306708	96.4%	95.9%	98.0%
Ele	Ele	6740	99193	98.9%	98.5%	99.9%	8896	148151	98.9%	99.7%	100.0%
Github	Github	4431	151355	98.6%	96.4%	98.8%	1327	50942	97.8%	94.0%	98.5%
Yirendai	Yirendai	4585	61356	98.1%	97.5%	99.2%	6760	64451	97.5%	97.1%	100.0%
Total		77278	4312686	71.7%	79.9%	90.7%	113020	2875337	75.7%	94.4%	96.6%

[1] 'Both' indicates the percentage of flows starting from a domain name query or containing an X.509 certificate.
[2] Applications in bold are added to the study to examine the classification effectiveness of applications from the same vendor.

TABLE IV
RANKING OF ATTRIBUTE OVERLAP BETWEEN APPLICATIONS

#	Applications		Domain	SNI	Average
1	Alipay	Taobao	11.31%	58.37%	34.84%
2	Taobao	Amap	26.18%	21.70%	23.94%
3	Taobao	Ele	11.57%	17.98%	14.78%
4	Baidu Search	Baidu Map	11.75%	17.46%	14.61%
5	Taobao	Weibo	12.96%	7.34%	10.15%
6	Amap	Weibo	9.80%	10.15%	9.98%
7	Facebook	Instagram	3.59%	15.78%	9.68%
8	Facebook	Airbnb	2.55%	15.01%	8.78%
9	Alipay	Amap	6.65%	10.73%	8.69%
10	Baidu Search	Yirendai	7.81%	9.13%	8.47%

TABLE V
STATISTICAL INFORMATION OF THE PUBLIC CROSS-PLATFORM DATASET

Dataset	#Flow	#Packet	#Application
Cross-Platform(Android)	27,846	656,044	215
Cross-Platform(iOS)	20,858	707,717	196

we can infer the performance lower bound of MPAF from the results of this experiment.

We conduct the experiments on a general computing server with 2*Intel® Xeon® Gold 6330 CPU @ 2.00GHz and DDR4 2933MHz 256GB memory.

2) Cross-Validation: We employ repeated random sub-sampling to mitigate the potential impact of accidental errors introduced by the dataset partitioning. We partition each experimental dataset into a training dataset and a validation dataset in a randomized manner using a 70-30 split and repeat this split ten times and compute the average of the resulting experimental results.

3) Evaluation Criteria: In our evaluation, we employ Precision (Prec.), Recall (Rec.), Accuracy (Acc.), F1_Macro (F1), and Average Waiting Time (AWT) as metrics to provide a comprehensive evaluation of the overall performance.

- **Precision**: We define the precision of application App_i as the ratio of correctly classified encrypted flows belonging to App_i to the total number of encrypted flows classified as App_i. The Precision is the macro average of all application precisions.
- **Recall**: We define the recall of Application App_i as the ratio of correctly classified encrypted flows belonging to App_i to the total number of encrypted flows that actually belong to App_i. The Recall is the macro average of all application recalls.
- **Accuracy**: We define accuracy as the overall ratio of correctly classified encrypted flows to the total number of encrypted flows.
- **F1_Macro**: We calculate the F-score of each application App_i as the harmonic mean of its Precision and Recall. The F1_Macro is the macro average of all F-scores.
- **AWT**: We define the waiting time WT_f of a flow f as the time required from the initialization of the flow until sufficient packets are collected or the flow is prematurely terminated to implement classification. The average waiting time (AWT) is the arithmetic mean of the waiting times. A smaller AWT value implies a quicker implementation of traffic classification.

B. Experimental Results of MPAF

1) The Parameters Selection for Leaf-Node Masking Tree-Based Classifier: Table VI shows the experimental results of the classifier parameter selection. We make a grid search for both the number of trees K and the classification threshold P. For each parameter combination, we evaluate the classification accuracy in each phase and perform summary calculations. Based on the Table VI, all parameter combinations show remarkably high classification accuracy for the summary of all phases. In some parameter combinations, the first two phases completely classify all flows, leaving none for the third phase. We use '-' in the table to indicate this. The parameter combinations $\{K = 20, P = 0.8\}$, $\{K = 30, P = 0.5\}$, and $\{K =$

TABLE VI
EXPERIMENTAL RESULTS OF CLASSIFIER PARAMETER SELECTION

Dataset:	$T = 10$ (# *of trees*)					$T = 20$ (# *of trees*)					$T = 30$ (# *of trees*)				
Manually Collected	$P = 0.4$	$P = 0.5$	$P = 0.6$	$P = 0.7$	$P = 0.8$	$P = 0.4$	$P = 0.5$	$P = 0.6$	$P = 0.7$	$P = 0.8$	$P = 0.4$	$P = 0.5$	$P = 0.6$	$P = 0.7$	$P = 0.8$
Domain Name Phase	99.93%	99.93%	**99.97%**	**99.97%**	**99.97%**	99.93%	99.94%	**99.97%**	**99.97%**	**99.97%**	99.94%	99.95%	**99.97%**	**99.97%**	**99.97%**
Handshake Metadata Phase	98.13%	98.37%	98.53%	98.82%	99.08%	98.24%	98.50%	98.64%	98.89%	99.16%	98.35%	98.54%	98.67%	98.91%	**99.17%**
Message Sequence Phase	73.50%	76.40%	74.67%	76.76%	79.42%	69.05%	77.10%	75.19%	78.48%	**80.14%**	65.68%	76.78%	76.52%	78.29%	80.10%
Summary of All Phases	99.32%	99.38%	99.38%	99.43%	99.47%	99.35%	99.41%	99.41%	99.45%	**99.50%**	99.38%	99.43%	99.42%	99.46%	99.49%

Dataset:	$T = 10$ (# *of trees*)					$T = 20$ (# *of trees*)					$T = 30$ (# *of trees*)				
Cross-Platform(Android)	$P = 0.4$	$P = 0.5$	$P = 0.6$	$P = 0.7$	$P = 0.8$	$P = 0.4$	$P = 0.5$	$P = 0.6$	$P = 0.7$	$P = 0.8$	$P = 0.4$	$P = 0.5$	$P = 0.6$	$P = 0.7$	$P = 0.8$
Domain Name Phase	98.92%	98.94%	99.05%	99.08%	99.10%	99.10%	99.10%	99.08%	99.09%	99.10%	99.10%	**99.13%**	99.08%	99.09%	99.10%
Handshake Metadata Phase	99.32%	99.31%	99.84%	99.91%	99.91%	99.55%	99.55%	99.86%	99.89%	99.91%	99.21%	99.32%	99.86%	99.91%	**99.93%**
Message Sequence Phase	-	7.14%	**43.61%**	39.65%	41.66%	-	-	42.25%	37.98%	37.49%	-	-	34.67%	36.20%	42.48%
Summary of All Phases	98.77%	98.71%	99.08%	99.07%	98.99%	98.99%	98.93%	**99.11%**	99.06%	98.99%	98.91%	98.88%	99.09%	99.05%	98.99%

Dataset:	$T = 10$ (# *of trees*)					$T = 20$ (# *of trees*)					$T = 30$ (# *of trees*)				
Cross-Platform(iOS)	$P = 0.4$	$P = 0.5$	$P = 0.6$	$P = 0.7$	$P = 0.8$	$P = 0.4$	$P = 0.5$	$P = 0.6$	$P = 0.7$	$P = 0.8$	$P = 0.4$	$P = 0.5$	$P = 0.6$	$P = 0.7$	$P = 0.8$
Domain Name Phase	95.52%	95.68%	95.92%	96.04%	96.11%	95.64%	95.92%	96.04%	96.08%	96.11%	95.72%	96.00%	96.08%	**96.12%**	96.11%
Handshake Metadata Phase	99.78%	99.78%	99.78%	99.89%	99.88%	99.78%	99.78%	99.77%	99.77%	99.88%	99.78%	99.78%	99.78%	99.89%	**100.00%**
Message Sequence Phase	31.31%	31.37%	39.24%	34.39%	39.49%	36.67%	43.59%	44.44%	**44.53%**	41.03%	30.80%	32.75%	37.95%	39.29%	42.92%
Summary of All Phases	96.33%	96.27%	96.36%	95.96%	95.45%	96.39%	96.39%	96.24%	95.96%	95.45%	96.39%	**96.51%**	96.33%	95.93%	95.48%

TABLE VII
CLASSIFICATION PERFORMANCE IN EACH PHASE

Dataset	**Manually Collected**			**Cross-Platform(Android)**			**Cross-Platform(iOS)**		
MPAF Phase	**Accuracy**	**AWT**	**Proportion**	**Accuracy**	**AWT**	**Proportion**	**Accuracy**	**AWT**	**Proportion**
Domain Name Phase	99.97%	0.00s	67.02%	99.08%	0.00s	80.38%	96.08%	0.00s	70.33%
Handshake Metadata Phase	98.67%	1.32s	32.45%	99.86%	0.92s	19.42%	99.78%	0.62s	28.34%
Message Sequence Phase	76.52%	4.19s	0.53%	34.67%	1.02s	0.19%	37.95%	1.00s	1.33%
Summary of All Phases	99.42%	0.45s	100.00%	99.09%	0.18s	100.00%	96.33%	0.19s	100.00%

TABLE VIII
CLASSIFICATION PERFORMANCE WITH A SINGLE PHASE

Datasets	**Manually Collected**				**Cross-Platform(Android)**				**Cross-Platform(iOS)**			
Phase	**Acc.**	**Prec.**	**Rec.**	**F1**	**Acc.**	**Prec.**	**Rec.**	**F1**	**Acc.**	**Prec.**	**Rec.**	**F1**
Domain Name Phase Only	77.88%	93.32%	75.97%	79.10%	89.96%	96.11%	86.95%	90.26%	76.85%	82.10%	74.15%	76.00%
Handshake Metadata Phase Only	99.01%	98.83%	98.90%	98.86%	98.99%	98.88%	98.84%	98.82%	99.03%	98.77%	98.83%	98.74%
Message Sequence Phase Only	98.06%	97.95%	97.95%	97.95%	86.35%	82.39%	79.73%	80.40%	68.48%	64.53%	63.64%	62.40%
Use All Phases	99.42%	99.32%	99.32%	99.32%	99.09%	98.75%	98.54%	98.58%	96.33%	96.09%	95.72%	95.66%

20, $P = 0.6$} achieve the best overall accuracy of 99.50%, 96.51%, and 99.11% for the three experimental datasets, respectively. From the perspective of attribute sources, the message sequence is less discriminative than the attributes in the first two phases and is therefore more susceptible to the classifier parameters. Considering the classification accuracy across three datasets, we select {$K = 30, P = 0.6$} as the final parameter combination.

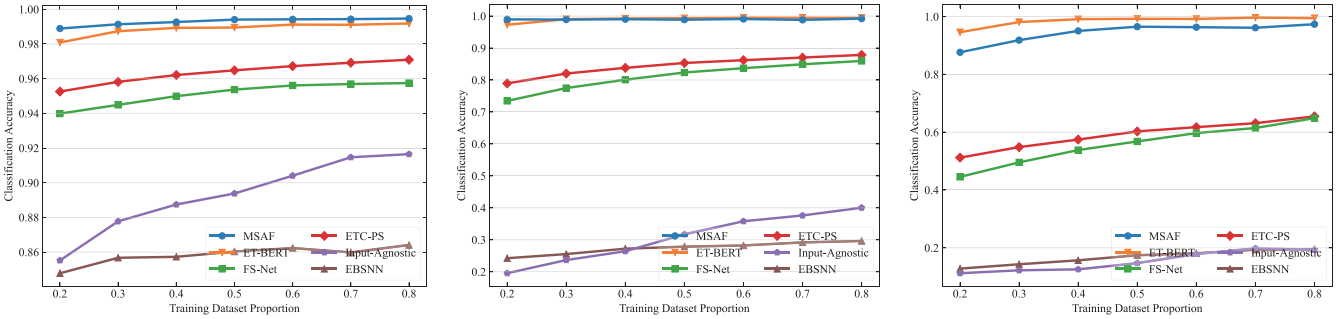
Table VII presents the classification performance in each phase under the optimal parameter combination, including classification accuracy, AWT, and the proportion of classified flows. The first two phases both demonstrate high classification accuracy and low AWT, and together handle approximately 99% of the flows. The flows classified in the message sequence phase are residual flows from the first two phases, introducing challenges to the classification process. The accuracy of the message sequence phase ranges from 34.67% to 76.52%, which is significantly lower than the first two phases. This phase tries to use message sequence features to classify them as accurately as possible, thereby improving the overall system accuracy. The AWT of the domain name phase is 0.00s because the associated DNS records are always generated and captured before the flow starts. The domain name phase of MPAF handles approximately 70% of the flows, which significantly reduces the waiting time for most flows. The AWT of the handshake metadata phase ranges from 0.62s to 1.32s, while the AWT of the message sequence phase ranges from 1.00s to 4.19s. The overall AWT of the three phases are 0.45, 0.18, and 0.19 seconds(s) for the three experimental datasets, respectively.

2) The Effectiveness of a Single MPAF Phase: Table VIII shows the classification performance of each single phase. We implement this experiment by skipping the prior phases and enforcing classification at a given phase. Based on the combined results of each phase across the three datasets, the domain name phase shows high precision but relatively lower recall. The handshake metadata phase achieves both high accuracy and recall. The performance of the message sequence phase fluctuates significantly, making it suitable as a supplementary fallback. In the manually collected dataset and the cross-platform(Android) dataset, the combined use of all three phases further enhances overall performance.

TABLE IX
COMPARISON RESULTS OF CLASSIFICATION ACCURACY

Datasets		Manually Collected					Cross-Platform(Android)					Cross-Platform(iOS)				
Methods	**# of Packets**	**Acc.**	**Prec.**	**Rec.**	**F1**	**AWT[1]**	**Acc.**	**Prec.**	**Rec.**	**F1**	**AWT**	**Acc.**	**Prec.**	**Rec.**	**F1**	**AWT**
FS-Net	32 raw packets	95.61%	95.37%	95.64%	95.61%	17.64s(39.20×)[2]	83.69%	76.46%	78.17%	76.74%	12.14s(67.44×)	59.68%	54.01%	57.29%	54.41%	11.12s(58.53×)
EBSNN	3 payload packets	86.23%	85.54%	86.19%	85.36%	0.41s(0.91×)	28.22%	14.49%	14.16%	18.50%	0.69s(3.83×)	18.10%	5.99%	4.60%	10.82%	0.41s(2.16×)
ET-BERT	5 payload packets	99.12%	99.01%	99.00%	99.01%	1.00s(2.22×)	99.52%	99.42%	99.50%	99.40%	0.87s(4.83×)	99.18%	98.92%	99.01%	98.95%	0.71s(3.74×)
ETC-PS	40 raw packets	96.72%	96.55%	96.55%	96.56%	20.86s(46.36×)	86.20%	80.26%	84.37%	79.22%	13.41s(74.50×)	61.74%	55.53%	59.34%	55.44%	14.73s(77.53×)
Input-Agnostic	32 raw packets	90.40%	90.52%	91.01%	90.38%	17.59s(39.09×)	35.79%	23.88%	26.29%	25.47%	12.21s(67.83×)	17.85%	8.94%	9.40%	11.96%	10.97s(57.74×)
MPAF	$\leq$ 12 messages	99.42%	99.32%	99.32%	99.32%	0.45s	99.09%	98.58%	98.75%	98.54%	0.18s	96.33%	95.66%	96.09%	95.72%	0.19s

[1] A smaller AWT value indicates a shorter waiting time for traffic classification.
[2] The values in brackets denote the improvement ratio in AWT of MPAF when compared to state-of-the-art approaches.



(a) Manually Collected (b) Cross-Platform(Android) (c) Cross-Platform(iOS)

Fig. 5. Classification accuracy with different training dataset proportions.

TABLE X
EXPERIMENTAL RESULTS OF CROSS-DATASET CLASSIFICATION

Application	FS-Net		EBSNN		ET-BERT		ETC-PS		Input-Agnostic		MPAF-D		MPAF-M		MPAF	
	Prec.	**Rec.**	**Prec.**	**Rec.**	**Prec.**	**Rec.**	**Prec.**	**Rec.**	**Prec.**	**Rec.**	**Prec.**	**Rec.**	**Prec.**	**Rec.**	**Prec.**	**Rec.**
Alipay	70.29%	70.74%	16.76%	10.01%	68.56%	33.92%	60.66%	46.45%	47.43%	53.49%	71.85%	43.87%	86.70%	**85.37%**	**94.75%**	80.67%
Taobao	11.49%	21.32%	27.15%	16.24%	64.55%	30.75%	29.44%	15.64%	14.95%	18.10%	69.65%	42.02%	75.70%	44.09%	**78.27%**	**83.15%**
Amap	55.99%	33.25%	19.23%	34.60%	22.42%	37.39%	43.27%	39.92%	37.57%	37.53%	61.17%	35.31%	**70.00%**	82.98%	62.50%	**92.08%**
Facebook	57.35%	67.62%	36.58%	5.72%	7.41%	10.83%	68.77%	62.79%	50.27%	52.50%	72.97%	49.56%	**88.81%**	83.06%	86.87%	**97.88%**
Instagram	64.30%	**88.79%**	40.20%	12.65%	0.00%	0.00%	40.30%	79.55%	59.68%	74.07%	83.34%	63.16%	89.48%	86.09%	**96.03%**	88.49%
Twitter	40.80%	87.39%	2.99%	0.15%	2.59%	1.37%	32.17%	**95.93%**	40.99%	64.66%	96.63%	70.97%	99.51%	86.32%	**99.84%**	71.07%
Weibo	14.33%	4.51%	13.63%	9.53%	60.70%	23.25%	6.00%	5.52%	16.07%	13.24%	23.68%	66.64%	32.61%	67.37%	**61.44%**	**73.62%**
Airbnb	77.54%	37.29%	8.54%	10.34%	29.30%	17.16%	61.17%	33.45%	35.58%	49.06%	53.90%	99.32%	70.50%	**99.84%**	**94.13%**	99.77%
Linkedin	**65.53%**	63.15%	4.04%	4.10%	11.02%	21.00%	61.62%	49.12%	27.22%	50.88%	17.81%	60.47%	28.71%	70.42%	53.91%	**81.37%**
Evernote	97.38%	33.15%	37.36%	4.28%	85.91%	20.39%	**99.75%**	21.29%	75.25%	26.88%	71.61%	34.63%	91.51%	90.84%	85.28%	**96.79%**
Blued	59.93%	64.96%	7.46%	6.82%	26.46%	44.95%	44.09%	**77.91%**	17.49%	11.74%	48.82%	32.21%	**66.61%**	38.47%	56.71%	64.22%
Ele	56.47%	22.46%	24.83%	57.07%	55.27%	52.56%	**73.17%**	35.29%	44.57%	30.06%	47.88%	**99.31%**	63.95%	78.92%	71.06%	78.99%
Github	32.46%	95.49%	43.27%	82.63%	31.77%	92.70%	35.17%	**98.22%**	46.92%	77.16%	86.24%	33.53%	**96.59%**	58.44%	63.32%	63.68%
Yirendai	17.03%	9.10%	10.07%	6.02%	42.36%	9.42%	88.67%	13.64%	44.79%	18.87%	69.27%	21.98%	95.69%	**43.48%**	**96.60%**	42.34%
Acc/F1	47.75%	45.18%	20.54%	17.24%	28.85%	27.82%	45.77%	43.15%	39.58%	37.88%	53.40%	51.94%	71.64%	70.43%	**79.09%**	**77.44%**

In the cross-platform(iOS) dataset, the overall performance is minimally affected by the accuracy of the domain name phase and the message sequence phase, but still remains relatively high.

VI. COMPARISONS WITH EXISTING APPROACHES

In this section, we conduct a comprehensive evaluation of MPAF by comparing it with five contemporary approaches in terms of classification accuracy, AWT, scalability of the dataset, and resilience to cross-dataset classification. Furthermore, we compare MPAF against one advanced approach that addresses the challenge of unknown application recognition.

A. Preliminary

1) Existing Approaches: In this section, we conduct a comparison analysis of MPAF with six state-of-the-art approaches for encrypted traffic classification. These approaches are selected for their superior declared classification performance, with a focus on incorporating a variety of technical routes.

- FS-Net [22] utilizes an encoder-decoder structure to generate features and directly classify the feature vectors. We use the parameters mentioned in their paper to rebuild the neural network using Pytorch.
- EBSNN [30] is a method that aggregates the payload segments of the first few packets of each flow as input and classifies them using an encoder with an attention mechanism. We utilize open-source code to implement the approach on our datasets.
- ET-BERT [31] is a pre-trained model on large-scale unlabeled traffic data. We refine the model on our datasets through fine-tuning using the open-source code.
- ETC-PS [18] builds traffic path signatures for the first N data packets of each flow and uses machine learning algorithms to implement the classifier. We use six sequence features declared in the paper to rebuild the system.
- Input-Agnostic [34] is a neural network method that is built upon a hierarchical attention mechanism that integrates information from packet level, flow level, and

TABLE XI
EXPERIMENTAL RESULTS IN THE PRESENCE OF UNKNOWN APPLICATIONS

(a) Scenario A (Unknown App: Baidu Map / Airbnb / Ele)

Method (Phase)	MPAF				MPAF-D			MPAF-M				AUMS
	Phase 1	Phase 2	Phase 3	Summary	Phase 1	Phase 2	Summary	Phase 1	Phase 2	Phase 3	Summary	Summary
Unknown Prec.[1]	-	-	-	**95.68%**	-	-	84.49%	-	-	-	90.53%	71.34%
Unknown Rec.[1]	-	-	-	**100.00%**	-	-	**100.00%**	-	-	-	**100.00%**	10.36%
Accuracy	98.78%	88.91%	82.14%	**95.43%**	95.75%	65.86%	92.23%	98.78%	89.04%	58.12%	94.45%	74.07%
F1-Macro	92.17%	80.43%	72.24%	**96.49%**	90.91%	74.99%	93.80%	92.17%	80.08%	61.48%	95.67%	70.62%

(b) Scenario B (Unknown App: Taobao, Github, Yirendai)

Method (Phase)	MPAF				MPAF-D			MPAF-M				AUMS
	Phase 1	Phase 2	Phase 3	Summary	Phase 1	Phase 2	Summary	Phase 1	Phase 2	Phase 3	Summary	Summary
Unknown Prec.	-	-	-	**94.16%**	-	-	86.06%	-	-	-	89.09%	92.45%
Unknown Rec.	-	-	-	**100.00%**	-	-	**100.00%**	-	-	-	**100.00%**	11.72%
Accuracy	97.18%	98.73%	88.46%	**97.09%**	99.54%	69.70%	96.03%	97.18%	98.77%	70.09%	96.28%	81.18%
F1-Macro	91.18%	86.57%	77.90%	**97.72%**	92.63%	77.01%	96.68%	91.18%	86.04%	64.46%	96.98%	77.80%

(c) Scenario C (Unknown App: Evernote, Facebook, Weibo)

Method (Phase)	MPAF				MPAF-D			MPAF-M				AUMS
	Phase 1	Phase 2	Phase 3	Summary	Phase 1	Phase 2	Summary	Phase 1	Phase 2	Phase 3	Summary	Summary
Unknown Prec.	-	-	-	**92.16%**	-	-	81.05%	-	-	-	86.55%	86.76%
Unknown Rec.	-	-	-	**100.00%**	-	-	**100.00%**	-	-	-	**100.00%**	23.22%
Accuracy	99.82%	91.55%	78.70%	**95.60%**	96.84%	75.00%	93.13%	99.82%	91.87%	60.21%	94.50%	77.00%
F1-Macro	92.65%	85.17%	85.29%	**96.42%**	91.52%	79.19%	94.47%	92.65%	90.23%	64.28%	95.62%	74.61%

(d) Scenario D (Unknown App: Amap, Instagram, Linkedin)

Method (Phase)	MPAF				MPAF-D			MPAF-M				AUMS
	Phase 1	Phase 2	Phase 3	Summary	Phase 1	Phase 2	Summary	Phase 1	Phase 2	Phase 3	Summary	Summary
Unknown Prec.	-	-	-	**92.65%**	-	-	75.71%	-	-	-	83.25%	80.55%
Unknown Rec.	-	-	-	**100.00%**	-	-	**100.00%**	-	-	-	**100.00%**	9.73%
Accuracy	99.87%	82.58%	77.08%	**93.50%**	94.26%	74.59%	91.17%	99.87%	82.83%	65.75%	92.58%	78.57%
F1-Macro	92.71%	86.97%	82.24%	**93.89%**	89.74%	82.00%	91.97%	92.71%	87.18%	74.73%	93.23%	75.70%

[1] We consider the unknown applications as a distinct class in the computation of "Accuracy" and "F1_Macro".

trace level. We implement the approach on our datasets using open-source code.

- AMUS [25] is a novel approach that identifies unknown applications by setting a threshold on the discriminator. We use the parameters mentioned in the paper to rebuild the unknown application identification system.

2) Setting of MPAF: According to the experimental results presented in § V, we select $\{K = 30, P = 0.6\}$ as the parameter combination, and use a maximum of 5 payload packets per flow in the second phase and a maximum of 12 messages per flow in the third phase.

3) Experimental Setting: In this study, we evaluate MPAF against state-of-the-art approaches on six key aspects, namely classification accuracy, F1_Macro, AWT, scalability of the dataset, resilience to cross-dataset classification, and recognition of unknown applications. To compare **classification accuracy, F1_Macro, and AWT**, we conduct experiments on the manually collected dataset and two cross-platform datasets. We also investigate **the scalability of the dataset** in relation to classification accuracy, covering a range from 20% to 80% of the manually collected dataset and two cross-platform datasets.

Moreover, we explore **the robustness of cross-dataset classification** by training on the automatically collected dataset and validating on the manually collected dataset. Finally, to evaluate the performance of MPAF in **the recognition of unknown applications**, we compare it with a state-of-the-art approach on the manually collected dataset.

B. Classification Performance and AWT

Table IX presents the comparison experimental results of both the classification effectiveness metric and the classification efficiency metric. In terms of classification effectiveness metric, Table IX shows that MPAF achieves high classification accuracies of 99.42%, 99.09%, and 96.33% on the three datasets, accompanied by consistently high F1_Macro scores. On the manually collected dataset, MPAF surpasses all the state-of-the-art approaches on the classification accuracy. In addition, MPAF and EBSNN lead the other methods on the AWT metric. On the cross-platform(Android) dataset, the classification accuracy of MPAF and ET-BERT is comparable and leads other methods, while the AWT of MPAF leads all the other approaches. On the cross-platform(iOS) dataset, MPAF is slightly inferior to ET-BERT in classification accuracy but performs 3.74 × faster than ET-BERT on the AWT metric. The slightly inferior performance of MPAF is caused by the domain name phase. Table VII shows that the performance of the domain name phase falls short compared to the handshake metadata phase. To further improve performance, it is feasible to incorporate the first packet information into the domain name phase to enhance its classification capability. The good performance of ET-BERT on the cross-platform dataset can be

attributed to its reliance on a pre-trained transformer model, which exhibits better generalization capabilities on datasets with fewer samples.

Discussion: The Average Waiting Time (AWT) is the arithmetic mean of the waiting time when sufficient packets have been captured or the flow is prematurely terminated. Thus, the AWT is primarily determined by the number of packets required by each method. Based on the results in Table IX, MPAF achieves the AWTs of 0.45s, 0.18s, and 0.19s on the three datasets without packet limitations, respectively, improves the AWT by up to 77.53 times compared to the state-of-the-art approach. MPAF requires a maximum of 5 payload packets in the second phase and a maximum of 12 messages in the third phase, resulting in lower AWTs compared to state-of-the-art approaches. EBSNN and ET-BERT require a small, fixed number of payload packets, resulting in relatively low AWT ranges of 0.41s to 0.69s and 0.71s to 1.00s, respectively. FS-Net, ETC-PS, and Input-Agnostic achieve classification with 32-40 raw packets, resulting in much higher AWTs.

C. Robustness to Scale of the Dataset

The classification accuracy of MPAF and five state-of-the-art approaches, under different proportions of the training dataset, is presented in Figure 5. For the manually collected dataset and cross-platform(Android) dataset, as the proportion of the training dataset increases, the improvement of FS-Net, ETC-PS, and Input-Agnostic is notably significant, which highlights the substantial dependence of these approaches on the scale of the training dataset. The accuracy of other approaches, MPAF, ET-BERT, and EBSNN, do not increase significantly as the proportion of the training dataset increases. For the manually collected dataset and cross-platform(android) dataset, the accuracy of ET-BERT, FS-Net, and ETC-PS is fairly high at 20% training dataset proportion, which indicates the robustness of these approaches to the training dataset's scale. MPAF shows consistently high accuracy from 20% to 80% of the training dataset proportion with very small differences. This demonstrates the strong tolerance of MPAF to the inadequacy of the training dataset, and we can significantly reduce the amount of training dataset while maintaining high accuracy. For the cross-platform(iOS) dataset, some approaches show a significant decrease in classification performance under all training dataset proportions, while MPAF and ET-BERT maintain a high classification accuracy with only 20% of the training dataset.

D. Resilience to Cross-Dataset Classification

Table X reports the results of a comparison study of approaches when applied to cross-dataset classification. Specifically, the classifiers are trained on the automatically collected dataset and tested on the manually collected dataset. We create two MPAF variants for comparison in cross-dataset classification and the recognition of unknown applications in the subsequent subsection.

- MPAF-D is a Dual-phase attribute fingerprint where the first phase exploits domain name and handshake metadata and the second phase uses all attributes.
- MPAF-M is a three-phase attribute fingerprint that incorporates Multiple attributes in its last two phases. Specifically, the first phase exploits domain names, the second phase uses domain name and handshake metadata, and the third phase uses all attributes.

The state-of-the-art approaches exhibit accuracy and F1_Macro ranging from 17.24% to 47.75%. This represents a decline of 47.86% to 71.19% when compared to the baseline classification results presented in Table IX. The dissimilarities between the two datasets stem from differences in the UI operation logic of the application when generating traffic, resulting in differences in dataset distribution. The independent nature of MPAF's utilized attributes, such as Domain Name and Metadata, from the traffic content enables it to perform accurate classification for most applications. This results in an accuracy of 79.09%, signifying a decrease of 20.33%, and an F1_Macro of 77.44%, reflecting a drop of 21.88%.

E. Recognition of Unknown Applications

In addition to providing traffic classification for known applications in the training dataset, we can make some modifications to MPAF to enable it to identify unknown applications in real-world scenarios. We alter the classification output mode from labels to classification probability vectors $v = [p_1, p_2, \ldots, p_N]$, where p_i is the classification probability to i-th application. In the last phase of MPAF and its variants, we set a probability threshold τ to label the flows whose maximum probability is lower than the threshold, that is, $max(v) < \tau$, to the unknown applications.

To evaluate the robustness of MPAF, MPAF-D, MPAF-M, and AMUS against unknown applications, we randomly select three applications as unknown and remove the corresponding traffic from the training dataset, but keep them in the validation dataset. We conduct the experiments on four scenarios with different randomly selected unknown applications to improve the confidence of the experimental results. In the experimental result analysis, we treat the unknown applications as a distinct class.

Table XI reports the experimental results of classifying encrypted traffic in the presence of unknown applications. Experimental results reveal that MPAF attains the highest classification accuracy, ranging from 93.50% to 97.09%, F1_Macro from 93.89% to 97.72%, an unknown application precision rate ranging from 92.16% to 95.68%, and an unknown application recall rate of 100.00% across four scenarios. The overall performance of MPAF-M is higher than MPAF-D, but weaker than MPAF. Compared to MPAF-M, MPAF-D removes a phase designed for domain attributes and maintains comparable overall performance to MPAF-M. This indicates that independently utilizing domain attributes in the initial phase does not result in an overall performance decrease; instead, it can reduce the AWT of the classification.

AMUS demonstrates a classification accuracy ranging from 74.07% to 81.18%, F1_Macro from 70.62% to 77.80%, an unknown application precision rate varying between 71.34% to 92.45%, and an unknown application recall rate spanning 9.73% to 23.22% across four scenarios. In comparison to the precision rate for unknown applications, the recall

rate for unknown applications is notably lower. This could be attributed to the ambiguous classification boundaries of known applications. In comparison to AMUS, MPAF exhibits superior performance with a 14.93% to 21.36% advantage in classification accuracy, an improvement of 18.19% to 25.87% in F1_Macro, a 1.71% to 24.34% increase in precision for unknown application precision rate, and a 76.78% to 90.27% boost in unknown application recall rate.

VII. Discussion and Conclusion

This study introduces an efficient encrypted traffic classification system named Multi-Phase Attribute Fingerprint (MPAF), which leverages three phases to exploit attributes emerging at different time periods. The experimental results clearly demonstrate the superiority of MPAF over state-of-the-art approaches in terms of classification efficiency and robustness. The two datasets used in our study were captured, respectively, by volunteers and through machine automation. There still exists a performance gap between the cross-dataset classification and single-dataset classification. In future research, we will delve into the diminishing effectiveness of attributes across different datasets to achieve comparable classification accuracy. These findings will significantly broaden the applicability of automatically collected datasets for replacing manually collected datasets.

References

[1] *App Store*. Accessed: Jan. 2024. [Online]. Available: https://www.apple.com/ios/app-store/

[2] *Google Play*. Accessed: Jan. 2024. [Online]. Available: https://play.google.com/store

[3] T. Dierks and E. Rescorla, "The transport layer security (TLS) protocol version 1.2," Internet Eng. Task Force, USA, Tech. Rep. rfc5246, 2008.

[4] E. Rescorla, "The transport layer security (TLS) protocol version 1.3," Internet Eng. Task Force, USA, Tech. Rep. rfc8446, 2018.

[5] Y. Wang, X. Yun, Y. Zhang, L. Chen, and T. Zang, "Rethinking robust and accurate application protocol identification," *Comput. Netw.*, vol. 129, pp. 64–78, Dec. 2017.

[6] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Robust smartphone app identification via encrypted network traffic analysis," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 1, pp. 63–78, Jan. 2018.

[7] M. Conti, Q. Q. Li, A. Maragno, and R. Spolaor, "The dark side(-channel) of mobile devices: A survey on network traffic analysis," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 2658–2713, 4th Quart., 2018.

[8] A. Acar et al., "Peek-a-boo: I see your smart home activities, even encrypted!" in *Proc. 13th ACM Conf. Secur. Privacy Wireless Mobile Netw.*, Jul. 2020, pp. 207–218.

[9] B. Charyyev and M. H. Gunes, "IoT event classification based on network traffic," in *Proc. IEEE Conf. Comput. Commun. Workshops*, Jul. 2020, pp. 854–859.

[10] S. Dong, Z. Li, D. Tang, J. Chen, M. Sun, and K. Zhang, "Your smart home can't keep a secret: Towards automated fingerprinting of IoT traffic," in *Proc. 15th ACM Asia Conf. Comput. Commun. Secur.*, Oct. 2020, pp. 47–59.

[11] R. Alshammari and A. N. Zincir-Heywood, "Investigating two different approaches for encrypted traffic classification," in *Proc. 6th Annu. Conf. Privacy, Secur. Trust*, Oct. 2008, pp. 156–166.

[12] M. Korczynski and A. Duda, "Markov chain fingerprinting to classify encrypted traffic," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, Apr. 2014, pp. 781–789.

[13] M. Shen, M. Wei, L. Zhu, and M. Wang, "Classification of encrypted traffic with second-order Markov chains and application attribute bigrams," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 8, pp. 1830–1843, Aug. 2017.

[14] A. Montieri, D. Ciuonzo, G. Bovenzi, V. Persico, and A. Pescapé, "A dive into the dark web: Hierarchical traffic classification of anonymity tools," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 3, pp. 1043–1054, Jul. 2020.

[15] T. van Ede et al., "FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, vol. 27, 2020, pp. 1–18.

[16] G. Aceto, G. Bovenzi, D. Ciuonzo, A. Montieri, V. Persico, and A. Pescapé, "Characterization and prediction of mobile-App traffic using Markov modeling," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 1, pp. 907–925, Mar. 2021.

[17] X. Ma et al., "Context-aware website fingerprinting over encrypted proxies," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2021, pp. 1–10.

[18] S.-J. Xu, G.-G. Geng, X.-B. Jin, D.-J. Liu, and J. Weng, "Seeing traffic paths: Encrypted traffic classification with path signature features," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2166–2181, Jun. 2022.

[19] J. Piet, D. Nwoji, and V. Paxson, "GGFAST: Automating generation of flexible network traffic classifiers," in *Proc. ACM SIGCOMM Conf.*, New York, NY, USA, 2023, pp. 850–866.

[20] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapè, "Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges," *IEEE Trans. Netw. Service Manage.*, vol. 16, no. 2, pp. 445–458, Jun. 2019.

[21] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapè, "MIMETIC: Mobile encrypted traffic classification using multimodal deep learning," *Comput. Netw.*, vol. 165, Dec. 2019, Art. no. 106944.

[22] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun.*, Apr. 2019, pp. 1171–1179.

[23] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapé, "Toward effective mobile encrypted traffic classification through deep learning," *Neurocomputing*, vol. 409, pp. 306–315, Oct. 2020.

[24] M. Lotfollahi, R. S. H. Zade, M. J. Siavoshani, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, 2020.

[25] J. Zhang, F. Li, F. Ye, and H. Wu, "Autonomous unknown-application filtering and labeling for DL-based traffic classifier update," in *Proc. IEEE Conf. Comput. Commun.*, Jul. 2020, pp. 397–405.

[26] G. Aceto, D. Ciuonzo, A. Montieri, A. Nascita, and A. Pescapè, "Encrypted multitask traffic classification via multimodal deep learning," in *Proc. IEEE Int. Conf. Commun.*, Aug. 2021, pp. 1–6.

[27] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapé, "DISTILLER: Encrypted traffic classification via multimodal multitask deep learning," *J. Netw. Comput. Appl.*, vols. 183–184, Jun. 2021, Art. no. 102985.

[28] A. Nascita, A. Montieri, G. Aceto, D. Ciuonzo, V. Persico, and A. Pescapé, "XAI meets mobile traffic classification: Understanding and improving multimodal deep learning architectures," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 4225–4246, Dec. 2021.

[29] A. Montieri, G. Bovenzi, G. Aceto, D. Ciuonzo, V. Persico, and A. Pescapè, "Packet-level prediction of mobile-app traffic using multitask deep learning," *Comput. Netw.*, vol. 200, Dec. 2021, Art. no. 108529.

[30] X. Xiao, W. Xiao, R. Li, X. Luo, H. Zheng, and S. Xia, "EBSNN: Extended byte segment neural network for network traffic classification," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3521–3538, Sep. 2022.

[31] X. Lin, G. Xiong, and G. Gou, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, 2022, pp. 633–642.

[32] M. Jiang et al., "Accurate mobile-app fingerprinting using flow-level relationship with graph neural networks," *Comput. Netw.*, vol. 217, Nov. 2022, Art. no. 109309.

[33] A. Nascita, A. Montieri, G. Aceto, D. Ciuonzo, V. Persico, and A. Pescapé, "Improving performance, reliability, and feasibility in multimodal multitask traffic classification with XAI," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 1267–1289, Jun. 2023.

[34] J. Qu et al., "An input-agnostic hierarchical deep learning framework for traffic fingerprinting," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 589–606.

[35] F. Cerasuolo et al., "MEMENTO: A novel approach for class incremental learning of encrypted traffic," *Comput. Netw.*, vol. 245, May 2024, Art. no. 110374.

[36] G. Aceto, D. Ciuonzo, A. Montieri, V. Persico, and A. Pescapé, "MIRAGE: Mobile-app traffic capture and ground-truth creation," in *Proc. 4th Int. Conf. Comput., Commun. Secur. (ICCCS)*, Oct. 2019, pp. 1–8.

[37] G. Bovenzi et al., "Benchmarking class incremental learning in deep learning traffic classification," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 1, pp. 51–69, Feb. 2024.

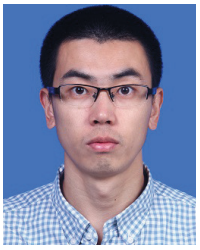
[38] (2019). *Monkeyrunner—Android Developers*. [Online]. Available: https://developer.android.com/studio/test/monkeyrunner/

[39] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "AppScanner: Automatic fingerprinting of smartphone apps from encrypted network traffic," in *Proc. IEEE Eur. Symp. Secur. Privacy*, Mar. 2016, pp. 439–454.

[40] J. Zhang and A. Moore, "Traffic trace artifacts due to monitoring via port mirroring," in *Proc. Workshop End-End Monitor. Techn. Services*, May 2007, pp. 1–8.

[41] P. V. Mockapetris, "Domain names: Concepts and facilities," Internet Eng. Task Force, USA, Tech. Rep. rfc1034, 1987.

Yige Chen received the Ph.D. degree in computer software and theory from the Institute of Information Engineering, Chinese Academy of Sciences (CAS), China, in 2022. He is currently a Lecturer with the College of Computer Science and Artificial Intelligence, Wenzhou University, China. His research interests include encrypted traffic fingerprints, network security, and machine learning.



Yipeng Wang (Senior Member, IEEE) received the Ph.D. degree in computer science from the Institute of Computing Technology, Chinese Academy of Sciences (CAS), China, in 2014. He is currently an Associate Professor with the College of Computer Science, Beijing University of Technology, China. He has published more than 50 research papers in refereed international journals and conferences, such as IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE International Conference on Network Protocols, and IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT. His research interests include networking, network security, and machine learning. He serves as a Program Committee Member for IJCAI 2021, IJCAI 2022, IJCAI 2023, and IJCAI 2024. He was a recipient of the Best Paper Award at IEEE International Conference on Network Protocols (ICNP) on his protocol format inference technology. He serves as a regular reviewer for IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING, IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING, and IEEE INTERNET OF THINGS JOURNAL.